

# IR-Board

---

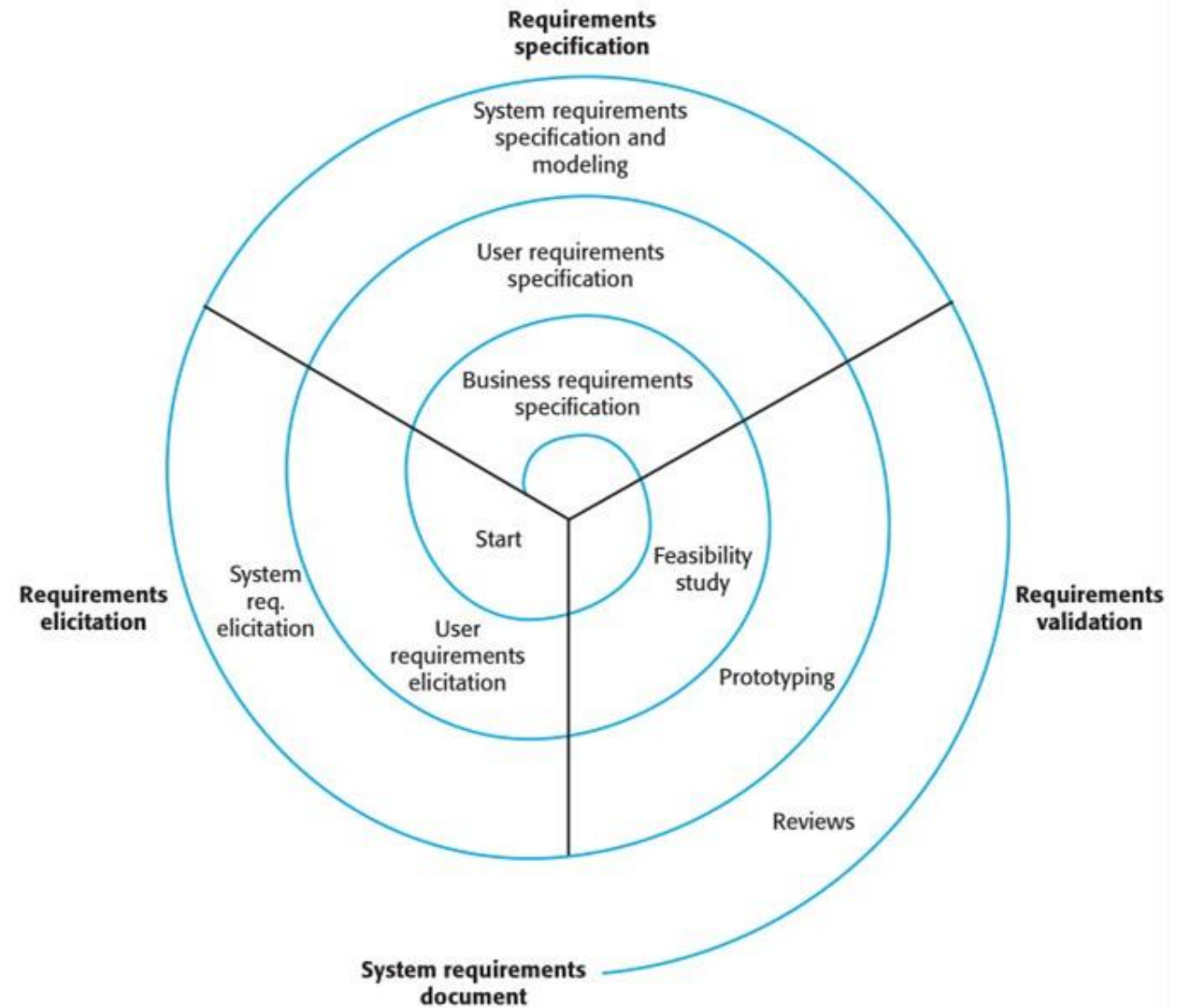
Requirements Management Platform  
Javier Carrasco Arango – UO294532

# Context

IR-Board is an MVP of a Requirements Management System

Several stakeholders – conflicting

IEEE 830 & ISO 29148

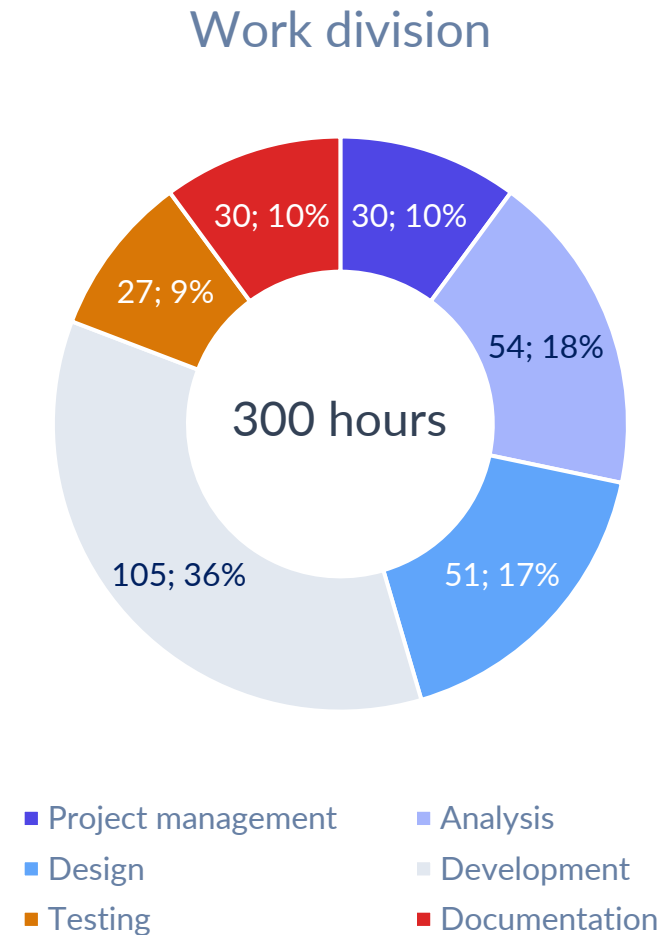


# Introduction

- Current tools follow either traditional requirements engineering or agile practices, and those who bridge the gap are proprietary and costly.
- Designed to provide:
  - Strong traceability and state management
  - Alignment with IEEE 830 and ISO 29148
  - Document management for agile or traditional artifacts
  - Serious security
  - Collaborative safeguards

# Scope and initial schedule

- Modules:
  - Security and access control
  - Requirement management
  - Project and project elements management
  - User management
  - Document management
  - Basic search and filtering
  - Entity Locking module.
- Extras:
  - Basic observability stack
  - SonarQube integration
  - Draw.io integration
  - Load testing
  - Manuals



# Budget

Created a fictional client's petition and fictional provider company to appropriately identify stakeholders, risks, and design the schedule and budget.

| Staff member          | Hourly rate<br>(without profit) |
|-----------------------|---------------------------------|
| Project manager       | 49,78 €                         |
| Service coordinator   | 46,75 €                         |
| Systems analyst       | 42,16 €                         |
| Technology consultant | 45,22 €                         |
| Software architect    | 49,15 €                         |
| Senior developer      | 43,85 €                         |
| Junior developer      | 38,55 €                         |

| IR-Board costs                               |                                      |             |
|----------------------------------------------|--------------------------------------|-------------|
| Cat. Num                                     | Category                             | Total       |
| 01                                           | IR-Board development costs breakdown | 13.026,52 € |
| 02                                           | Other                                | 172,50 €    |
| Total                                        |                                      | 13.199,02 € |
| Profit (25%)                                 |                                      | 3.299,75 €  |
| Amount to be increased on the first category |                                      | 3.472,25 €  |
| Total hours                                  |                                      | 300,00      |
| Dilution increase per billable hour          |                                      | 11,58 €     |

| Client budget overview |                    |
|------------------------|--------------------|
| Project management     | 1.840,86 €         |
| Analysis               | 2.932,53 €         |
| Design                 | 2.863,95 €         |
| SonarQube setup        | 166,28 €           |
| Development            | 5.602,80 €         |
| Testing                | 1.364,15 €         |
| Documentation          | 1.729,94 €         |
| <b>Total</b>           | <b>16.500,52 €</b> |

Complete theoretical provider's financial situation over on appendix 13.1.1

# Analysis

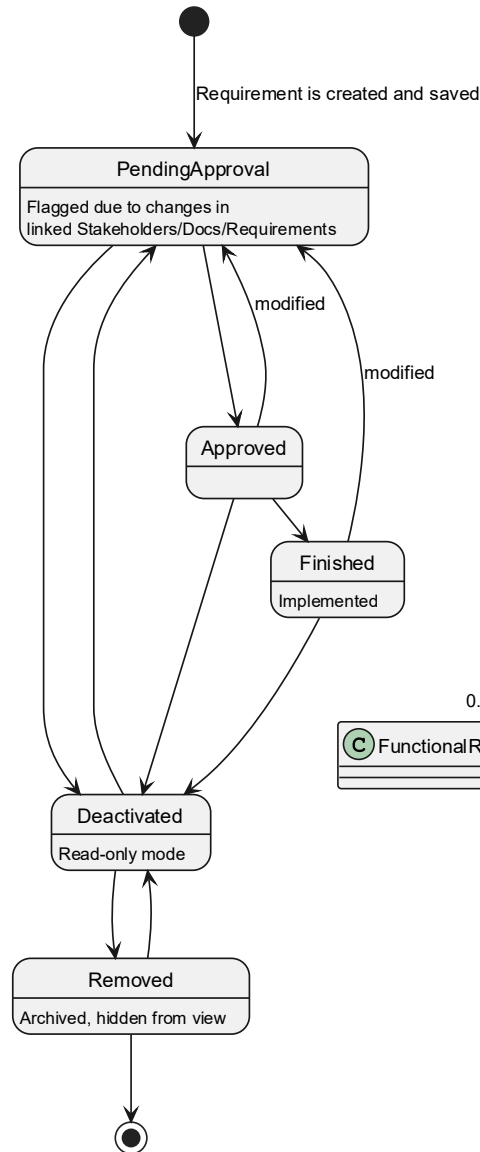
ReBAC differentiates the following roles:

- System-wide administrator
- Project manager
- Requirement engineer
- Stakeholder

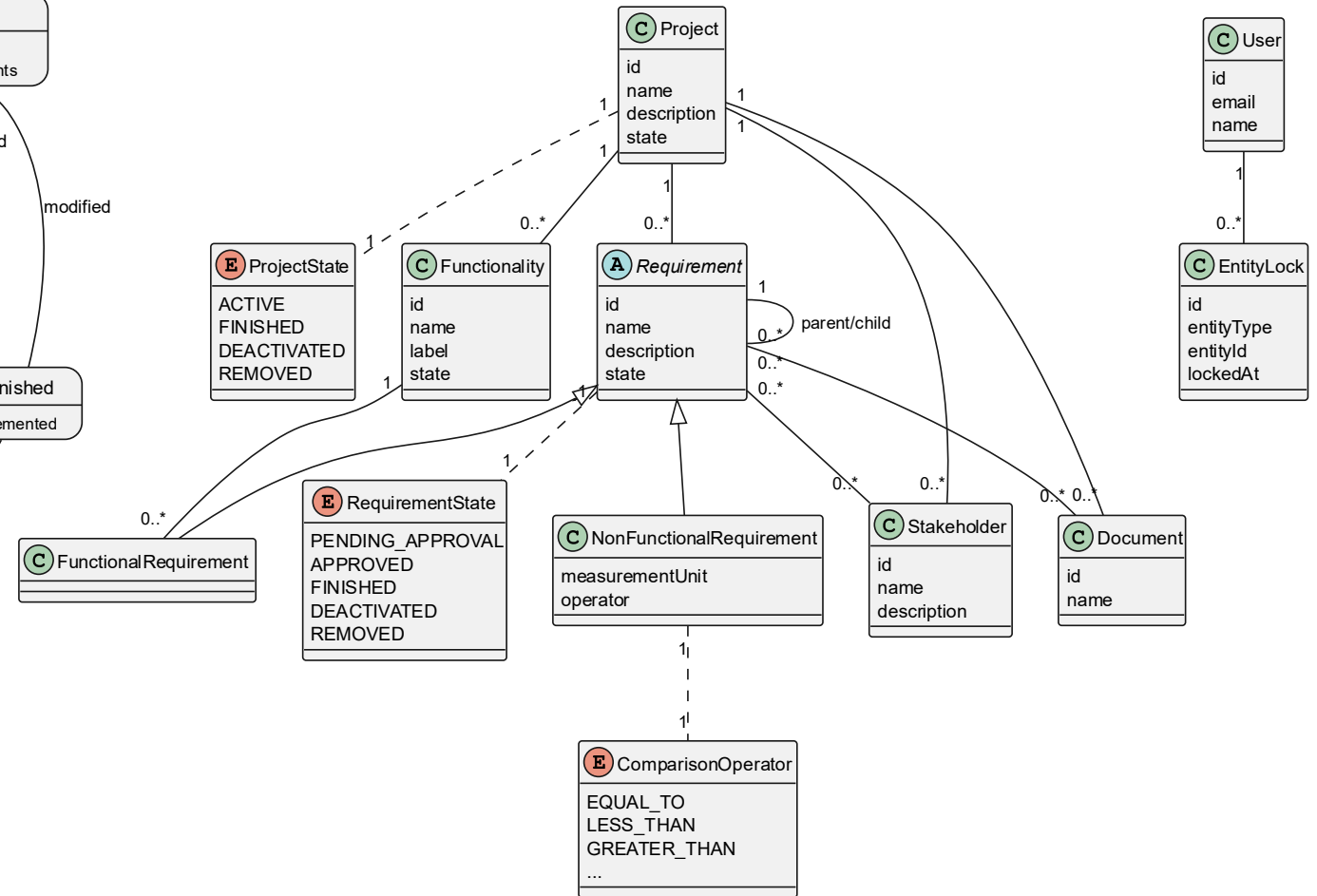
The highest permission prevails.

Traceability via dynamic identifier, & identity slug.

Lifecycle of a Requirement (IR-Board)



Backend Domain Diagram



# Architecture & technologies

Spring Boot vs Express.js and FastAPI

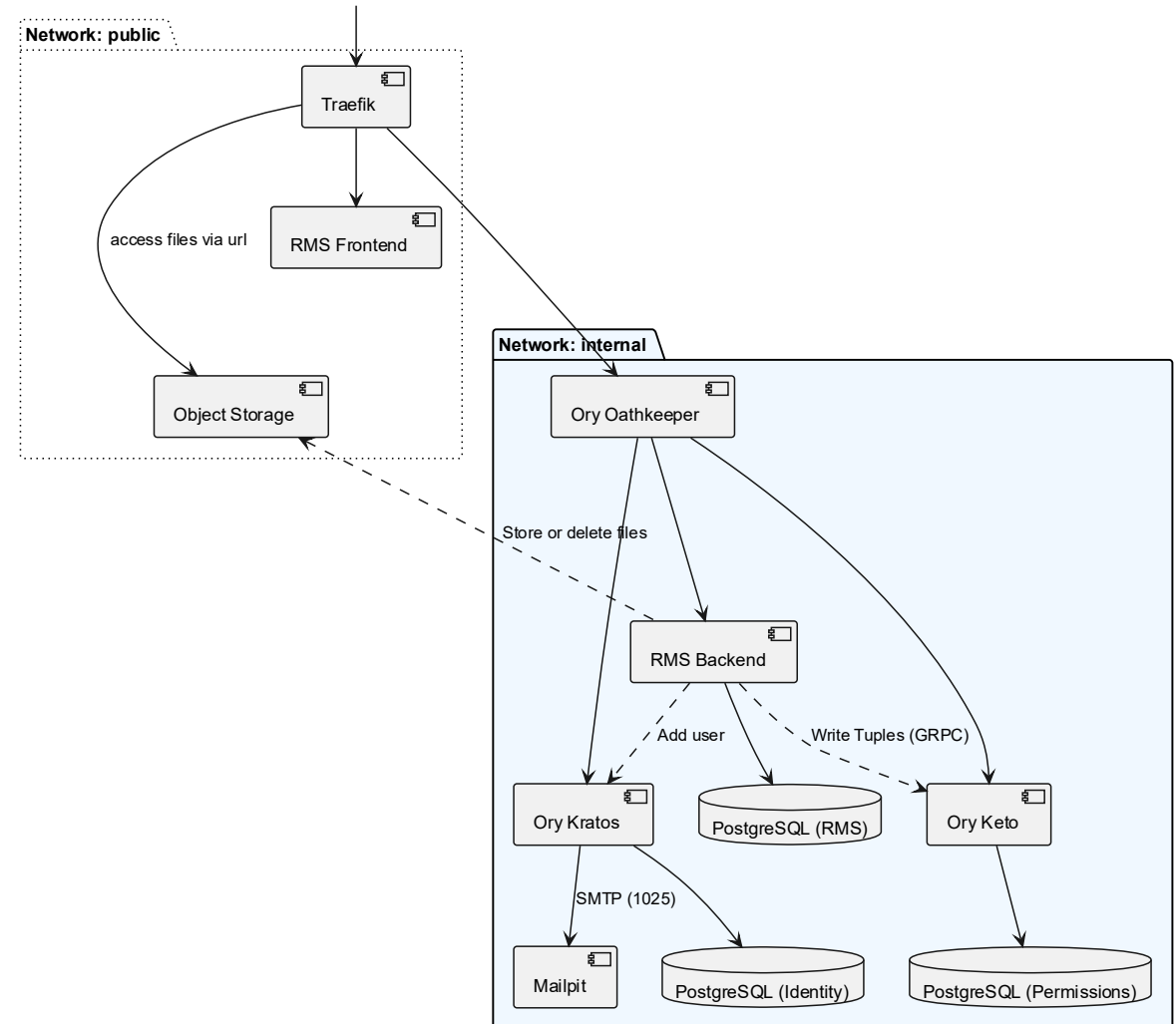
- Familiarity and strong conventions
- Database integration capabilities
- Personal preference for java

React + Typescript vs Angular

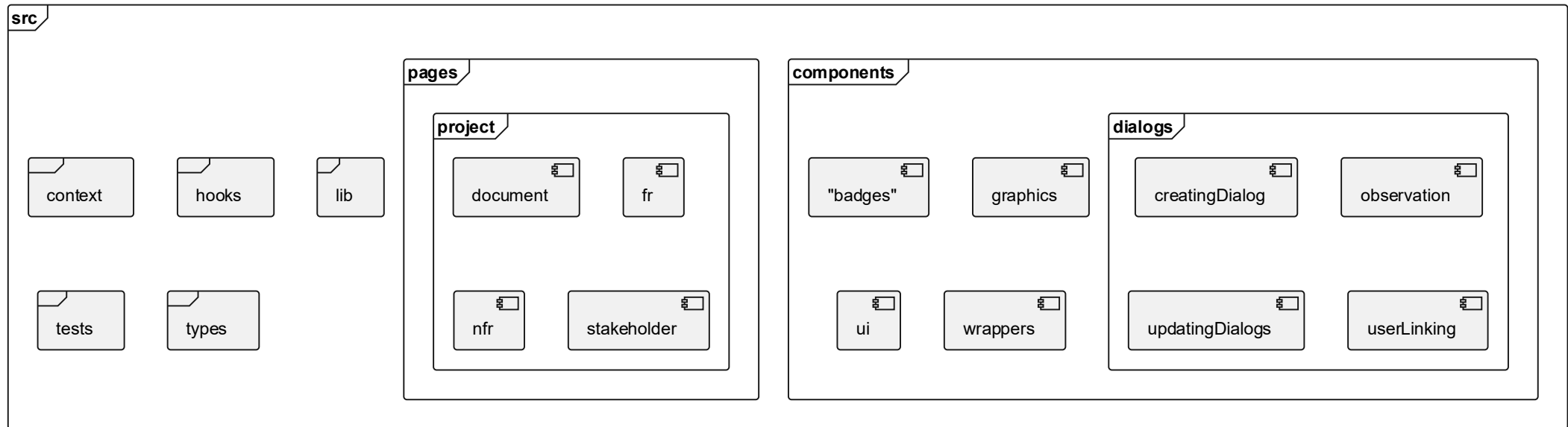
- Moderate learning curve, already familiar
- High flexibility

PostgreSQL vs MongoDB

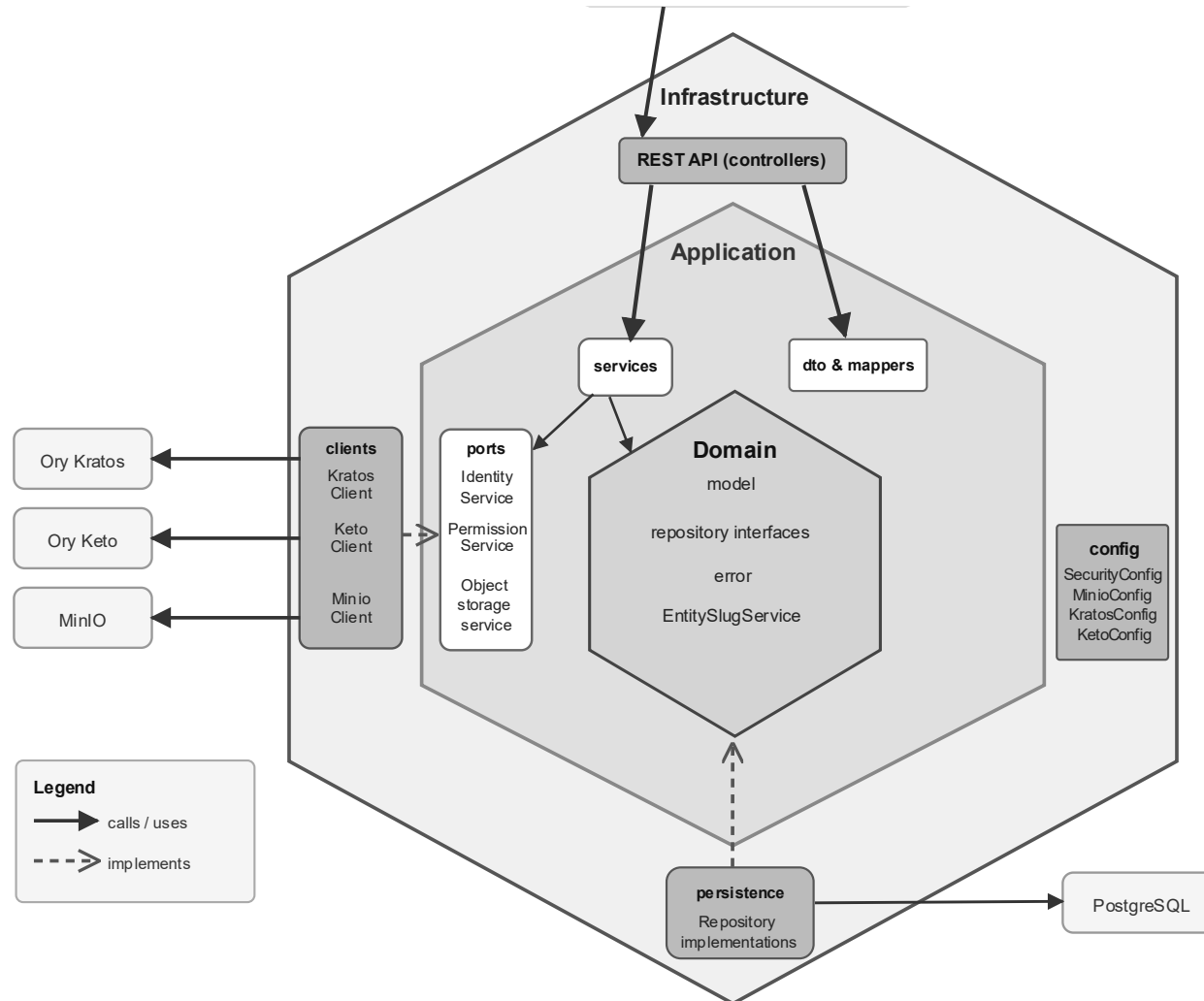
- Native support from Ory ecosystem
- Unlikely to change schema
- Out of the box traceability



# Architecture & technologies

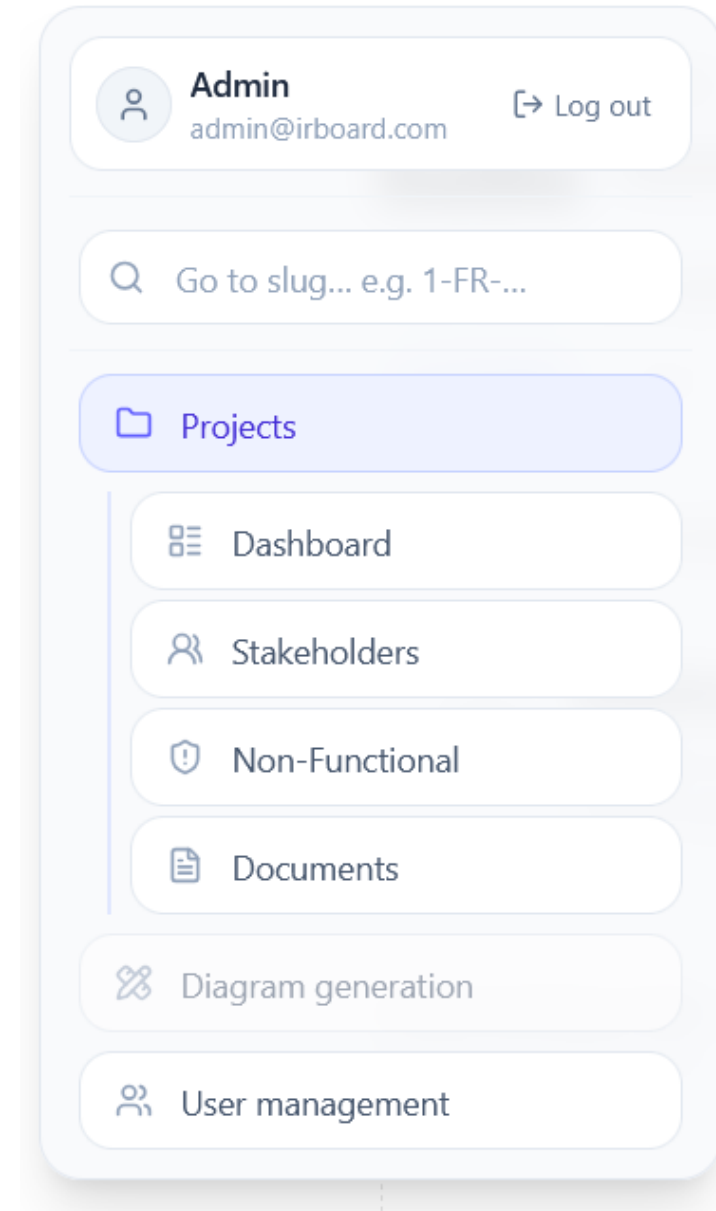


# Architecture & technologies

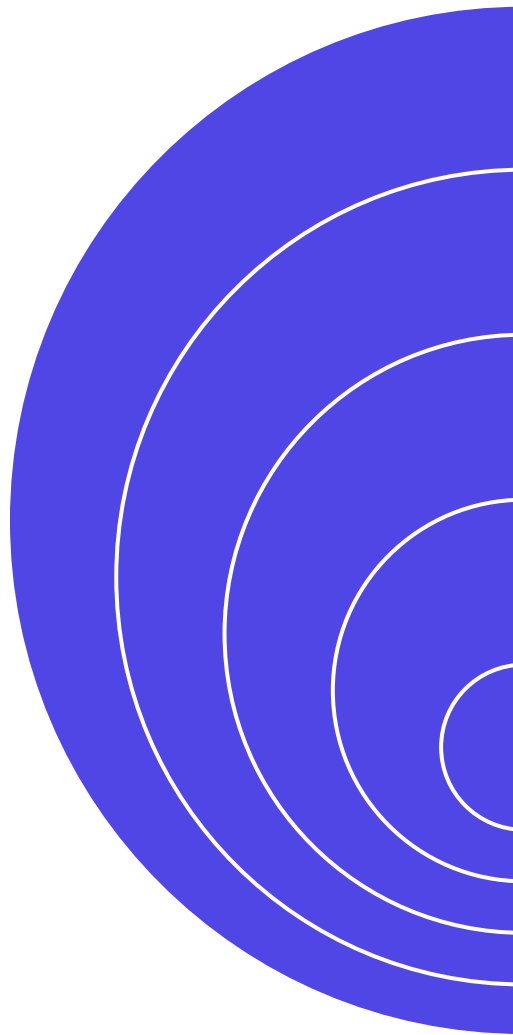


# Design process

- Frontend
  - Hand-drawn sketches
  - Navigability diagram
- Minimalism, good contrast, neutral palette
- Reveal information via exploration
- Every main design decision was checked with the tutors/clients



# Testing



|                                   |                                                                                                                           |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Unit testing & quality gate       | <ul style="list-style-type: none"><li>• JUnit5 + Mockito</li><li>• 80% coverage, no critical code smells</li></ul>        |
| Integration & Acceptance          |                                                                                                                           |
| E2E for full user journeys        | <ul style="list-style-type: none"><li>• Playwright, limited scope</li></ul>                                               |
| Load testing                      | <ul style="list-style-type: none"><li>• Gatling, 500 concurrent users with no request &gt;0.7s</li></ul>                  |
| Accessibility & usability testing | <ul style="list-style-type: none"><li>• Lighthouse audits, targeting 80+</li><li>• 4 usability testing sessions</li></ul> |
|                                   |                                                                                                                           |
|                                   |                                                                                                                           |
|                                   |                                                                                                                           |
|                                   |                                                                                                                           |

# Risk analysis – Mitigations

## **Risk: Modification of weekly working schedule**

Project management: Schedule – 0.81

Controlled redistribution of working hours to follow an acceptable weekly workload.

## **Risk: Erroneous identification of requirements**

Technical: Requirements – 0.45

Close involvement with the project tutors during identification and periodic alignment meetings.

## **Opportunity: Benefits gained through Ory**

Technical: Security / Architecture – 0.39

Align system design with native flows.

## **Risk: Lack of experience with the Ory**

Technical: Complexity and interfaces – 0.28

Allocation of self study hours and early prototyping of the Ory ecosystem integrations.

## **Risk: Incompatibility of design with Ory**

Technical: Architecture / Integration – 0.42

Introduce an abstraction layer between the application domain and the ecosystem services.

# Issues encountered

## Infrastructure complexity

- Deciding which services to place behind Oathkeeper

## SonarQube/CI coverage bug

- CI ran standard tests, not coverage-instrumented ones
- No test coverage reached SonarQube

## Document upload technical debt

- The uploads are routed through the backend.
- Documented as future work.

## Ory domain requirement

- Most of its features are built around cookies
- Required domain purchase for deployment

## Typst nested-list compatibility

- Plugin was used to avoid switching technologies

## Other

- MinIO dependency went into maintenance mid-project, future work
- Time constraints caused cut decisions

# Results

## Schedule deviations

**Total effort:**  
300h -> 310h (+3%)  
**Testing:**  
27h -> 35h (+29%)  
**Project man.:**  
30h -> 24h (-20%)  
  
**Analysis:** +6%  
**Design:** +4%  
**Documentation:** -7%

## Budget deviation

Development cost breakdown deviated +3%  
  
Actual profit:  
25% -> ~22% (-11%)  
  
No extra budget or contingency reserve was needed.

## Weekly schedule risk

- Materialized but controlled

## Erroneous requirements identification

- Did not materialize

## Lack of Ory/ReBAC experience

- Materialized but low impact, contained

## Ory workflow incompatibility

- Partially materialized, the abstraction from the driver allowed for workarounds

## Ory integration opportunity

- Taken advantage of

## Typst risk and opportunities

- Unaccounted for, materialized
- Improved documentation speed

# Future work and conclusions

## What was achieved



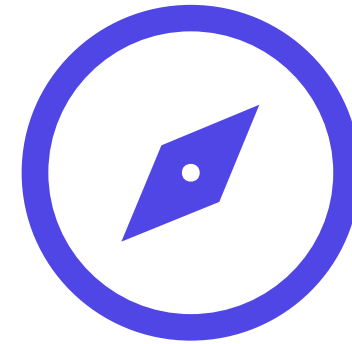
- Full lifecycle coverage
- Serious zero-trust perimeter
- Full containerized stack

## Lessons learned



- Trust boundaries
- Architecture patterns
- Self-managed execution

## Future work



- Hardening for production
- Testing/UX extension
- Feature completion

# Demo

